

Notebook: decsion_tree/kfold_validation_iris.ipynb

[--- MARKDOWN CELL ---]

K-Fold Cross Validation with Decision Tree

This notebook implements 5-fold cross validation on the Iris dataset using sklearn's DecisionTreeClassifier.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import cross_val_score, KFold, train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[--- CODE CELL ---]

```
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

Load and Explore the Iris Dataset

[--- CODE CELL ---]

```
# Load Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

print(f"Feature names: {iris.feature_names}")
print(f"Target names: {iris.target_names}")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
```

[--- CODE CELL ---]

```
# Create DataFrame for exploration
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = pd.Categorical.from_codes(y, iris.target_names)
df.head()
```

[--- CODE CELL ---]

```
df.describe()
```

[--- CODE CELL ---]

```
# Visualize class distribution
plt.figure(figsize=(8, 5))
sns.countplot(x='species', data=df)
plt.title("Class Distribution in Iris Dataset")
plt.xlabel("Species")
plt.ylabel("Count")
plt.show()
```

[--- CODE CELL ---]

```
# Pairplot to visualize feature relationships
sns.pairplot(df, hue='species', diag_kind='kde')
plt.suptitle("Iris Dataset - Feature Relationships", y=1.02)
```

```
plt.show()
```

[--- MARKDOWN CELL ---]

K-Fold Cross Validation (k=5)

[--- CODE CELL ---]

```
# Set up K-Fold cross validation with k=5
k = 5
kfold = KFold(n_splits=k, shuffle=True, random_state=42)

# Create Decision Tree classifier
dt_classifier = DecisionTreeClassifier(random_state=42)

# Perform cross validation
cv_scores = cross_val_score(dt_classifier, X, y, cv=kfold, scoring='accuracy')

print(f"K-Fold Cross Validation (k={k}) Results:")
print(f"Individual Fold Scores: {cv_scores}")
print(f"Mean Accuracy: {cv_scores.mean():.4f}")
print(f"Standard Deviation: {cv_scores.std():.4f}")
```

[--- CODE CELL ---]

```
# Visualize fold scores
plt.figure(figsize=(10, 6))
folds = [f'Fold {i+1}' for i in range(k)]
colors = ['steelblue' if score == max(cv_scores) else 'coral' for score in cv_scores]

bars = plt.bar(folds, cv_scores, color=colors, edgecolor='black')
plt.axhline(y=cv_scores.mean(), color='green', linestyle='--', linewidth=2, label=f'Mean: {cv_scores.mean():.4f}')
plt.xlabel("Fold")
plt.ylabel("Accuracy")
plt.title(f"5-Fold Cross Validation Scores - Decision Tree on Iris")
plt.ylim(0.8, 1.05)
plt.legend()
plt.grid(True, alpha=0.3, axis='y')

# Add value labels on bars
for bar, score in zip(bars, cv_scores):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{score:.4f}', ha='center', va='bottom', fontsize=10)

plt.tight_layout()
plt.show()
```

[--- MARKDOWN CELL ---]

Manual K-Fold Implementation

[--- CODE CELL ---]

```
# Manual implementation to understand K-Fold better
fold_accuracies = []
fold_train_sizes = []
fold_test_sizes = []

print("Manual K-Fold Cross Validation:")
print("=" * 60)

for fold_idx, (train_idx, test_idx) in enumerate(kfold.split(X)):
    # Split data for this fold
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    # Train model
    dt = DecisionTreeClassifier(random_state=42)
    dt.fit(X_train, y_train)

    # Evaluate
```

```

accuracy = dt.score(X_test, y_test)
fold_accuracies.append(accuracy)
fold_train_sizes.append(len(train_idx))
fold_test_sizes.append(len(test_idx))
print(f"Fold {fold_idx + 1}:")
print(f"  Train samples: {len(train_idx)}, Test samples: {len(test_idx)}")
print(f"  Accuracy: {accuracy:.4f}")
print()
print("=" * 60)
print(f"Mean Accuracy: {np.mean(fold_accuracies):.4f} (+/- {np.std(fold_accuracies):.4f})")

```

[--- MARKDOWN CELL ---]

Hyperparameter Tuning with K-Fold CV

[--- CODE CELL ---]

Test different max_depth values

```

max_depths = [1, 2, 3, 4, 5, None]
results = []

```

```

for depth in max_depths:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    scores = cross_val_score(dt, X, y, cv=kfold, scoring='accuracy')
    results.append({
        'max_depth': str(depth) if depth else 'None',
        'mean_accuracy': scores.mean(),
        'std': scores.std()
    })

```

```

results_df = pd.DataFrame(results)
print("Hyperparameter Tuning Results:")
print(results_df.to_string(index=False))

```

[--- CODE CELL ---]

Plot hyperparameter tuning results

```

plt.figure(figsize=(10, 6))
x_pos = range(len(results_df))
plt.bar(x_pos, results_df['mean_accuracy'],
        yerr=results_df['std'], capsize=5,
        color='steelblue', edgecolor='black', alpha=0.8)

plt.xticks(x_pos, results_df['max_depth'])
plt.xlabel("max_depth")
plt.ylabel("Mean Accuracy")
plt.title("Decision Tree - 5-Fold CV Accuracy vs max_depth")
plt.ylim(0.8, 1.05)
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

```

[--- MARKDOWN CELL ---]

Train Final Model and Visualize

[--- CODE CELL ---]

Train on full dataset

```

final_dt = DecisionTreeClassifier(max_depth=3, random_state=42)
final_dt.fit(X, y)

```

Visualize the decision tree

```

plt.figure(figsize=(20, 10))
plot_tree(final_dt,
          feature_names=iris.feature_names,
          class_names=iris.target_names,

```

```

        filled=True,
        rounded=True,
        fontsize=10)
plt.title("Decision Tree for Iris Classification (max_depth=3)")
plt.tight_layout()
plt.show()

```

[--- CODE CELL ---]

```

# Feature importance
feature_importance = final_dt.feature_importances_
sorted_idx = np.argsort(feature_importance)[::-1]
plt.figure(figsize=(10, 6))
plt.barh([iris.feature_names[i] for i in sorted_idx], feature_importance[sorted_idx], color='steelblue')
plt.xlabel("Feature Importance")
plt.title("Decision Tree Feature Importance - Iris Dataset")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

```

[--- MARKDOWN CELL ---]

Conclusion

Using 5-fold cross validation on the Iris dataset with a Decision Tree classifier:

- We obtained a mean accuracy of approximately 95-96%
- The standard deviation across folds is relatively low, indicating stable performance
- Hyperparameter tuning showed that `max_depth=3` provides good accuracy while preventing overfitting
- The most important feature for classification is 'petal width'